

SNOWPRO® ADVANCED: DATA ENGINEER (DEA-C02) EXAM STUDY GUIDE

Last Updated: August 22, 2025

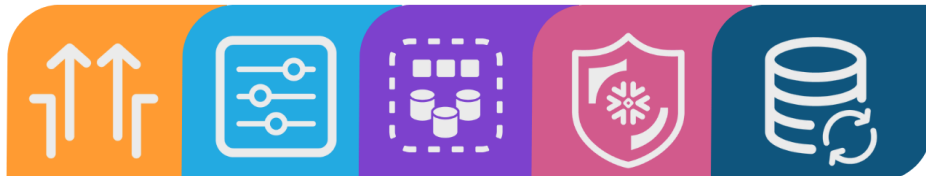


TABLE OF CONTENTS

SNOWPRO ADVANCED: DATA ENGINEER STUDY GUIDE OVERVIEW	2
RECOMMENDATIONS FOR USING THE GUIDE	2
SNOWPRO ADVANCED: DATA ENGINEER CERTIFICATION OVERVIEW	3
SNOWPRO ADVANCED: DATA ENGINEER PREREQUISITE	4
STEPS TO SUCCESS	4
SNOWPRO ADVANCED: DATA ENGINEER SUBJECT AREA BREAKDOWN	4
Domain 1.0: Data Movement	5
Domain 1.0 Study Resources	6
Domain 2.0: Performance Optimization	7
Domain 2.0 Study Resources	7
Domain 3.0: Storage & Data Protection	8
Domain 3.0 Study Resources	8
Domain 4.0: Data Governance	9
Domain 4.0 Study Resources	9
Domain 5.0: Data Transformation	10
Domain 5.0 Study Resources	10
SNOWPRO ADVANCED: DATA ENGINEER SAMPLE QUESTIONS	11
NEXT STEPS	14
REGISTERING FOR YOUR EXAM	14
MAINTAINING YOUR CERTIFICATION	14

SNOWPRO ADVANCED: DATA ENGINEER STUDY GUIDE OVERVIEW

This study guide highlights concepts that may be covered on Snowflake's SnowPro Advanced: Data Engineer Certification exam.

Use of this study guide does not guarantee certification success.

Holding the SnowPro Core certification in good standing is a prerequisite for taking the Advanced: Data Engineer certification.

For an overview and more information on the SnowPro Core Certification exam or SnowPro Advanced Certification series, please navigate [here](#).

RECOMMENDATIONS FOR USING THE GUIDE

This guide will show the Snowflake topics and subtopics covered on the exam. Following the topics will be additional resources consisting of videos, documentation, blogs, and/or exercises to help you understand data engineering with Snowflake.

Estimated length of time required to complete the study guide: 10 – 13 hours.

Some links may have more value than others, depending on your experience. The same amount of time should not be spent on each link. Some links may appear in more than one content domain.

SNOWPRO ADVANCED: DATA ENGINEER CERTIFICATION OVERVIEW

The **SnowPro Advanced: Data Engineer** certification tests advanced knowledge and skills used to apply comprehensive data engineering principles using Snowflake. The exam will assess skills through scenario-based questions and real-world examples.

This certification will test the ability to:

- Source data from Data Lakes, APIs, and on-premises
- Transform, replicate, and share data across cloud platforms
- Design end-to-end near real-time streams
- Design scalable compute solutions for Data Engineer workloads
- Evaluate performance metrics

Target Audience:

2 + years of data engineering experience, including practical experience using Snowflake for Data Engineer tasks. In addition, successful candidates may have:

- A working knowledge of Restful APIs, SQL, semi-structured datasets, and cloud native concepts.
- Programming experience is a plus.

This exam is designed for:

- Data Engineers
- Software Engineers

SNOWPRO ADVANCED: DATA ENGINEER PREREQUISITE

Eligible individuals must hold an active SnowPro Core Certified credential. If you feel you need more guidance on the fundamentals, please see the [SnowPro Core Exam Study Guide](#).

STEPS TO SUCCESS

1. [Review the Data Engineer Exam Guide](#)
2. [Attend Snowflake's Instructor-Led Data Engineering I Training](#)
3. [Attend Snowflake's Instructor-Led Data Engineer II Training](#)
4. [Review and study applicable white papers and documentation](#)
5. [Get hands-on practical experience with relevant business requirements using Snowflake](#)
6. [Attend Snowflake Webinars](#)
7. [Attend Snowflake Virtual Hands-on Labs for practical experience](#)
8. [Practice with sample questions here](#)
9. [Schedule your exam](#)
10. Take your exam!

Additional Snowflake Asset to check out for Data Engineering:

[Cloud Data Engineering for Dummies](#)

SNOWPRO ADVANCED: DATA ENGINEER SUBJECT AREA BREAKDOWN

The following table contains the domains and weightings covered on the exam. It is not a comprehensive listing of all the content that will be presented on the exam.

Domain	Domain Weightings
1.0 Data Movement	26%
2.0 Performance Optimization	21%
3.0 Storage and Data Protection	14%
4.0 Data Governance	14%
5.0 Data Transformation	25%



Domain 1.0: Data Movement

1.1 Given a data set, load data into Snowflake.

- Outline considerations for data loading
- Define data loading features and potential impact

1.2 Ingest data of various formats through the mechanics of Snowflake.

- Required file formats
- Ingestion of structured, semi-structured, and unstructured data
- Implementation of stages and file formats

1.3 Troubleshoot data ingestion.

- Identify causes of ingestion errors
- Determine resolutions for ingestion errors

1.4 Design, build, and troubleshoot continuous data pipelines.

- Stages
- Tasks
- Streams
- Snowpipe (for example, Auto ingest as compared to Rest API)
- Snowpipe Streaming

1.5 Analyze and differentiate types of data pipelines.

- Create User-Defined Functions (UDFs)
- Design and use the Snowflake SQL API
- Create data pipelines in Snowpark

1.6 Install, configure, and use connectors to connect to Snowflake.

- Kafka connectors
- Spark connectors
- Python connectors

1.7 Design and build data sharing solutions.

- Implement a data share
- Create and manage views
- Implement row-level filtering
- Share data using the Snowflake Marketplace
- Share data using a listing

1.8 Outline when to use external tables and define how they work.

- Manage external tables
- Manage Iceberg tables
- Perform general table management
- Manage schema evolution
- Unload data

Domain 1.0 Study Resources

Lab Guides

[Accelerating Data Engineering with Snowflake & dbt](#)
[Auto-Ingest Twitter Data into Snowflake](#)
[Automating Data Pipelines to Drive](#)

Additional Assets

[Moving from On-Premises ETL to Cloud-Driven ELT \(white paper\)](#)
[Intro to Data Engineering with Snowpark for Python \(virtual hands-on lab\)](#)
[Building Continuous Data Pipelines with Snowflake \(virtual hands-on lab\)](#)

Snowflake Documentation Links

[Bulk loading - Local File Systems](#)
[COPY_HISTORY](#)
[COPY INTO](#)
[CREATE EXTERNAL TABLE](#)
[CREATE FILE FORMAT](#)
[CREATE STREAM](#)
[CREATE TASK](#)
[Data Loading Tutorials](#)
[Data Sharing](#)
[Databases, Tables & Views](#)
[DESCRIBE STAGE](#)
[Dynamic Tables](#)
[External Tables](#)
[File Functions](#)
[HASH](#)
[Internal stage parameters](#)
[Kafka and Spark Connectors](#)
[Loading Data into Snowflake](#)
[Python Connector](#)
[SYSTEM\\$PIPE_STATUS](#)
[Snowflake SQL Data Types](#)
[Snowpipe](#)
[Snowpipe REST API](#)
[Snowpipe Streaming](#)
[Streams and Tasks](#)
[VALIDATE - Data loading](#)
[VALIDATE_PIPE_LOAD](#)



➤ Domain 2.0: Performance Optimization

2.1 Troubleshoot underperforming queries.

- Identify underperforming queries
- Outline telemetry around the operation
- Increase efficiency
- Identify the root cause

2.2 Given a scenario, configure a solution for the best performance.

- Scale out compared to scale up
- Virtual warehouse properties (for example, size, multi-cluster)
- Query complexity
- Micro-partitions and the impact of clustering
- Materialized views
- Search optimization service

- Query acceleration service
- Snowpark-optimized warehouses
- Caching features

2.3 Monitor continuous data pipelines.

- Snowflake objects
 - Tasks
 - Streams
 - Snowpipe Streaming
 - Alerts
- Notifications
- Data Quality and data metric function monitoring

Domain 2.0 Study Resources

Lab Guides

[Resource Optimization: Performance](#)

[Resource Optimization: Usage Monitoring](#)

Additional Assets

[Performance Impact from Local and Remote](#)

[Disk Spilling \(blog\)](#)

[Snowflake: Visualizing Warehouse](#)

[Performance \(blog\)](#)

[Virtual Warehouse Best Practices \(blog\)](#)

Snowflake Documentation Links

[Account Usage](#)

[Configuring tasks to send notifications](#)

[COPY_HISTORY](#)

[Data quality and data metric functions](#)

[Databases, Tables & Views](#)

[LOAD_HISTORY View](#)

[Monitor query activity with QUERY HISTORY](#)

[Monitoring data loading](#)

[PIPE_USAGE_HISTORY view](#)

[Queries](#)

[QUERY_HISTORY, QUERY_HISTORY_BY_*](#)

[SHOW STREAMS](#)

[System clustering information](#)

[System functions](#)

[SYSTEM\\$CLUSTERING_INFORMATION](#)

[TASK_HISTORY](#)

[Using persisted query results](#)

[Using the query acceleration Service](#)

[Virtual Warehouses](#)



Domain 3.0: Storage & Data Protection

3.1 Implement and manage data recovery features in Snowflake.

- Time Travel
 - Impact of streams
- Fail-safe
- Cross-region and cross-cloud replication

3.2 Use system functions to analyze micro-partitions.

- Clustering depth
- Cluster keys

3.3 Use Time Travel and cloning to create new development environments.

- Clone objects
- Validate changes before promoting
- Rollback changes

Domain 3.0 Study Resources

Lab Guides

[Getting Started with Time Travel](#)

Snowflake Documentation Links

[Cloning considerations](#)

[Clustering keys and clustered tables](#)

[CREATE STREAM](#)

[Database replication and failover/failback](#)

[Account replication](#)

[Micro-partitions and data clustering](#)

[Parameters](#)

[Snowflake Time Travel & Fail-safe](#)

[Stage, pipe, and load history replication](#)

[SYSTEM\\$CLUSTERING_DEPTH](#)

[SYSTEM\\$CLUSTERING_INFORMATION](#)

[SYSTEM\\$EXPLAIN_PLAN_JSON](#)

[TABLE_STORAGE_METRICS](#) view



➤ Domain 4.0: Data Governance

4.1 Monitor data.

- Apply object tagging and classifications
- Use data classification to monitor data
- Manage data lineage and object dependencies
- Monitor data quality

4.2 Establish and maintain data protection.

- Implement column-level security
 - Use in conjunction with Dynamic Data Masking
 - Use in conjunction with external tokenization
 - Use projection policies
- Use data masking with Role-Based Access Control (RBAC) to secure sensitive data
- Explain the options available to support row-level security using Snowflake row access policies
 - Use aggregation policies
- Use DDL to manage Dynamic Data Masking and row access policies
- Use best practices to create and apply data masking policies
- Use Snowflake Data Clean Rooms to share data
 - Use the web-app
- Use the Snowflake developer API

Domain 4.0 Study Resources

Additional Assets

[Snowflake RBAC Security Prefers Role Inheritance to Role Composition](#) (blog)

Snowflake Documentation Links

[ALTER DYNAMIC TABLE](#)

[Continuous Data Protection](#)

[CREATE MATERIALIZED VIEW](#)

[Data access](#)

[Managing Governance in Snowflake](#)

[Managing Security in Snowflake](#)

[Object tagging](#)

[Projection policies](#)



Domain 5.0: Data Transformation

5.1 Define User-Defined Functions (UDFs) and outline how to use them.

- Snowpark UDFs (for example, Java, Python, Scala)
- Secure UDFs
- SQL UDFs
- JavaScript UDFs
- User-Defined Table Functions (UDTFs)
- User-Defined Aggregate Functions (UDAFs)

5.2 Define and create external functions.

- Secure external functions
- Work with external functions

5.3 Design, build, and leverage stored procedures.

- Snowpark stored procedures (for example, Java, Python, Scala)

5.4 Handle and transform semi-structured data.

- SQL Scripting stored procedures
- JavaScript stored procedures
- Transaction management

- Traverse and transform semi-structured data to structured data
- Transform structured data to semi-structured data

5.5 Handle and process unstructured data.

- Use unstructured data
 - URL types
- Use directory tables
- Use the Rest API

5.6 Use Snowpark for data transformation.

- Understand Snowpark architecture
- Query and filter data using the Snowpark library
- Perform data transformations using Snowpark (for example, aggregations)
- Manipulate Snowpark DataFrames

Domain 5.0 Study Resources

Additional Assets

[Best Practices for Managing Unstructured Data \(white paper\)](#)

Snowflake Documentation Links

[ARRAY_AGG](#)
[CREATE API INTEGRATION](#)
[CREATE EXTERNAL FUNCTION](#)
[CREATE PROCEDURE](#)
[Databases, Tables & Views](#)
[External Functions](#)
[Handler writing in Snowpark](#)

OBJECT_CONSTRUCT

[Queries](#)
[Semi-Structured Data](#)
[Snowflake Scripting](#)
[Snowpark API](#)
[Stored Procedures](#)
[Transactions](#)
[TRY_PARSE_JSON](#)
[UDFs \(User-Defined Functions\)](#)
[Unstructured data](#)
[Working with DataFrames](#)

SNOWPRO ADVANCED: DATA ENGINEER SAMPLE QUESTIONS

1. A Data Engineer needs to reload data into a table from files that have been in cloud storage; some files have been stored more than 64 days.

How should the data be loaded to avoid duplicating data that is already stored in the table?

- A. Execute a `COPY INTO <table>` command with the `FORCE` option set to `= TRUE`.
 - B. Delete all rows from the table and then execute a `COPY INTO <table>` command.
 - C. Execute a `COPY INTO <table>` command with the `LOAD_UNCERTAIN_FILES` option set to `= TRUE`.
 - D. Truncate the table and then execute a `COPY INTO <table>` statement.
2. A Data Engineer discovered that 9 days ago an outage caused a failure of a data pipeline run that was supposed to deliver data summarizing daily sales. The data did not update correctly and data is missing. The company is using a Snowflake Enterprise edition, with all data protection settings at the default values.

The Engineer needs to verify that the pipeline architecture is compliant with these data protection rules for data in the `FENIX_DWH` database:

1. 10 days of data history must be accessible in the `SALES` schema
2. 1 day of data history must be accessible in all other schemas

How can these requirements be met MOST cost effectively?

- A. Rely on the default Snowflake settings for data protection.
- B. Use the `ALTER ACCOUNT SET DATA_RETENTION_TIME_IN_DAYS = 10;` parameter.
- C. Use the `ALTER SCHEMA SALES SET DATA_RETENTION_TIME_IN_DAYS = 10;` parameter.
- D. Use the `ALTER DATABASE FENIX_DWH SET DATA_RETENTION_TIME_IN_DAYS = 10;` parameter.

3. A Data Engineer wants to parse JSON in Snowflake and run the following query:

```
create or replace temporary table vartab (id integer, v
varchar);
```

```
insert into vartab (id, v) values
(1, '[-1, 12, 289, 2188, false,]'),
(2, '{ "x" : "abc", "y" : false, "z": 10 } '),
(3, '[ "x" : "def", "y" : true, "z": 11 ] '),
(4, '[-1, 12, 289, 2188], NULL');
```

```
select id, try_parse_json(v)
from vartab
order by id;
```

What will be the result of the query?

A. The following error message will be returned: Error parsing JSON: incomplete object value.

B.

1	[-1, 12, 289, 2188, false, undefined]
2	{ "x": "abc", "y": false, "z": 10 }
3	NULL
4	NULL

C.

1	[-1, 12, 289, 2188, false, undefined]
2	{ "x": "abc", "y": false, "z": 10 }
3	[3, ["x" : "def", "y" : true, "z": 11]
4	NULL

D.

1	[-1, 12, 289, 2188, false, undefined]
2	{ "x": "abc", "y": false, "z": 10 }
3	NULL
4	[-1, 12, 289, 2188], NULL

4. A task was created to refresh data that is used to generate reports about data in a table. The task was running well until a large amount of historical data was added to the table.

Recently, queries to generate reports are failing. The ETL team checked the task history and reports:

1. The task has been executed successfully.
2. Once initiated the task resumes correctly.
3. The task has all required grants.
4. This is a stand-alone task with no dependencies.
5. The task is not using a stream.

Why is this task failing and what can be done to address this problem?

- A. The task is limited to a default duration of 60 minutes. The task needs to be recreated with the value `USER_TASK_TIMEOUT_MS` set to a higher value.
 - B. The task is limited to a default duration of 120 minutes. The task needs to be recreated with the value `USER_TASK_MINIMUM_TRIGGER_INTERVAL_IN_SECONDS` set to a higher value.
 - C. The task is limited to a default duration of 60 minutes. The task needs to be recreated with the value `USER_TASK_MINIMUM_TRIGGER_INTERVAL_IN_SECONDS` set to a higher value.
 - D. The task is limited to a default duration of 120 minutes. The task needs to be recreated with the value of `USER_TASK_TIMEOUT_MS` set to a higher value.
5. A Data Engineer recreated a table, `SALES`, using the `CREATE [ OR REPLACE ] TABLE` command without backing up the existing table. The Engineer found this information in the `QUERY HISTORY` view in the `INFORMATION SCHEMA`:

```
statement identifier for the "create or replace table" command:
1e5d0ca9-003e-22e6-b858-a7f5b37c5729
Time of execution: Wed, 26 Jun 2024 09:20:00 -0700
```

How can the original table data be retrieved?

- A. `DELETE FROM SALES;`
`INSERT INTO SALES SELECT * FROM SALES AT (TIMESTAMP => 'Wed, 26 Jun 2024 09:20:00 -0700'::timestamp_tz);`
- B. `UNDROP TABLE SALES;`
`DELETE FROM SALES;`
`INSERT INTO SALES SELECT * FROM SALES AT (OFFSET => -60*5);`
- C. `ALTER TABLE SALES RENAME TO SALES_new;`
`UNDROP TABLE SALES;`
- D. `ALTER TABLE SALES RENAME TO SALES_new;`
`UNDROP TABLE SALES;`
`INSERT INTO SALES SELECT * FROM SALES BEFORE (STATEMENT => '1e5d0ca9-003e-22e6-b858-a7f5b37c5729');`

Correct responses for sample questions:

1: d, 2: c, 3: b, 4: a , 5: c

NEXT STEPS

REGISTERING FOR YOUR EXAM

When you are ready to register for the exam navigate [here](#) to get started. Select the exam you want to take and click “Register Now”. This will take you to our Certification Management system where you will register to take the exam.

MAINTAINING YOUR CERTIFICATION

All Snowflake Certifications expire two (2) years after your certification issue date.

SnowPro Certifications can now be recertified through the Snowflake Continuing Education (CE) program which includes these options -

- Completion of eligible Snowflake [Instructor Led \(ILT\) Training Courses](#)
- Earning of an equivalent or higher-level SnowPro Certification

Note: You must have a valid Certification to participate in the Continuing Education (CE) program.

The information provided in this study guide is provided for your purposes only and may not be provided to third parties.

IN ADDITION, THIS STUDY GUIDE IS PROVIDED “AS IS”. NEITHER SNOWFLAKE NOR ITS SUPPLIERS MAKES ANY OTHER WARRANTIES, EXPRESS OR IMPLIED, STATUTORY OR OTHERWISE, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, TITLE, FITNESS FOR A PARTICULAR PURPOSE OR NONINFRINGEMENT.